CLIENT HANDOFF PACKAGE

# Fernhill Order Sync

HubSpot ↔ Supabase order and contact integration

---

Prepared by **Media & Software Manger**

Version 1.1 · 22 October 2026

Live demo: fernhill.mediaandsoftwaremanager.com

**Demo notice.** Fernhill Supply Co. is a fictional company. The live system holds only synthetic data: generated names, @example.com email addresses and seeded orders. This document contains no passwords, tokens or keys.

# Contents


**1. Overview** 4
- What it does 4
- Business value 4
- Live links 4
- Screenshots 5

**2. Architecture** 7
- The big picture 7
- The three components 7
- How data flows 7
- Why it is safe and reliable 8
- Technical appendix 9

**3. Setup guide: deploying from zero** 10
- Before you start 10
- Step 1: Get the code 10
- Step 2: Supabase (database) 11
- Step 3: Cloudflare Worker and web address 11
- Step 4: HubSpot app 11
- Step 5: Configure and deploy the web app 12
- Step 6: Connect the database to the web app 12
- Step 7: Load demo data and add the card 12
- Step 8: End-to-end check 12
- Every environment variable 13

**4. Operations runbook** 15
- Daily monitoring (2 minutes) 15
- What the daily job does 15
- Common issues 16
- Demo reset 17
- Rotating secrets 17

**5. Access and ownership** 18
- Accounts and services 18
- Secrets and keys 18
- Transferring ownership 19
- Transfer checklist 20

**6. Testing and QA** 21
- What the unit tests cover (54) 21
- What the database checks cover (77) 21
- Continuous integration and secret scanning 22
- Live smoke test results 22
- How to re-test after any change 23

**7. Limitations and next steps** 24
Free-tier constraints 24
Demo safeguards to remove or change for real data 24
What changes for real customer data 25
Upgrade roadmap (prioritised) 25
**8. Handoff checklist and sign-off** 26
Functionality 26
Reliability and safety 26
Quality evidence 26
Ownership and documentation 26
Sign-off 27

# 1. Overview

**Fernhill Order Sync** connects HubSpot CRM to a company's own order database and website, so the sales team sees order history inside HubSpot, and contact changes made in HubSpot reach the database automatically.

> **Demo notice.** Fernhill Supply Co. is a fictional company. The live system holds only synthetic data (generated names, `@example.com` email addresses, seeded orders) and refuses to store anything else.

## What it does

| Capability | What the user sees |
|---|---|
| **Orders inside HubSpot** | An *Orders* card on every contact record lists that contact's orders from the database, newest first. |
| **Order totals on the contact** | Each contact shows total orders, lifetime value and last order date, kept up to date automatically. |
| **HubSpot → database** | Editing a contact's name, email, phone or lifecycle stage in HubSpot updates the database within seconds. |
| **Website → HubSpot** | A public quote form creates (or reuses) the HubSpot contact and creates a deal for the request. Submitting twice never creates duplicates. |
| **Visible operations** | A public activity page lists every sync event, with no personal data, and counts any jobs that need attention. |

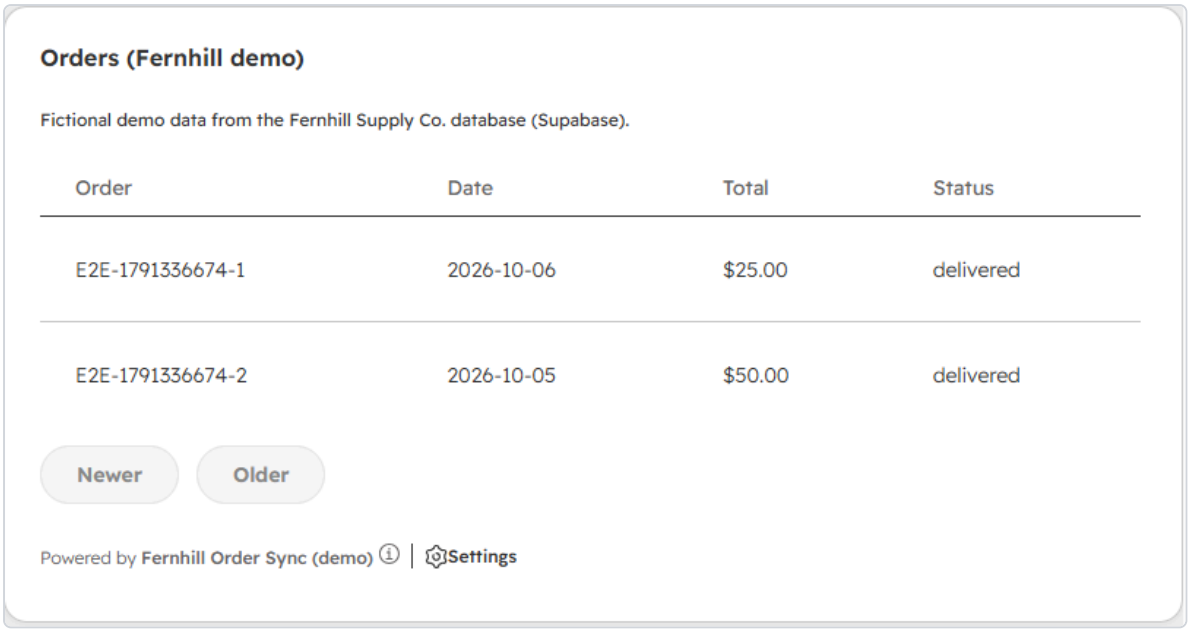
## Business value

- **One view of the customer.** Sales and support see orders without logging into another system.
- **No double entry.** Contact details are maintained once, in HubSpot; the database follows.
- **Reliable by design.** Every change is queued, retried automatically, and checked again by a daily job. Nothing loops and nothing is duplicated. This was verified on the live system on 9 October 2026 (see Testing and QA).
- **Low running cost.** The demo runs entirely on free tiers (HubSpot free CRM, Supabase Free, Cloudflare Workers Free). For production use with real customers, see Limitations and next steps.

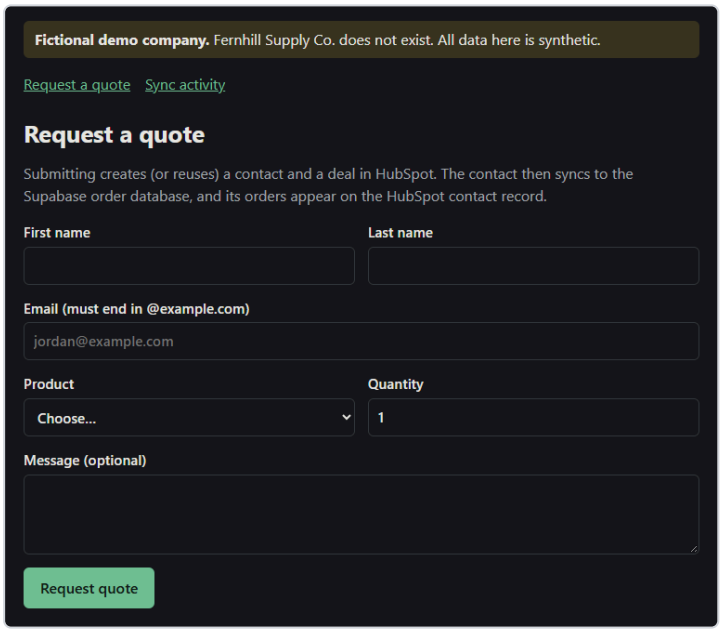
## Live links

- Quote form: https://fernhill.mediaandsoftwaremanager.com
- Sync activity: https://fernhill.mediaandsoftwaremanager.com/activity
- Demo video (90 seconds): `Fernhill-Order-Sync-Demo.mp4`, included in this delivery
- Source code: the `source/` folder, included in this delivery

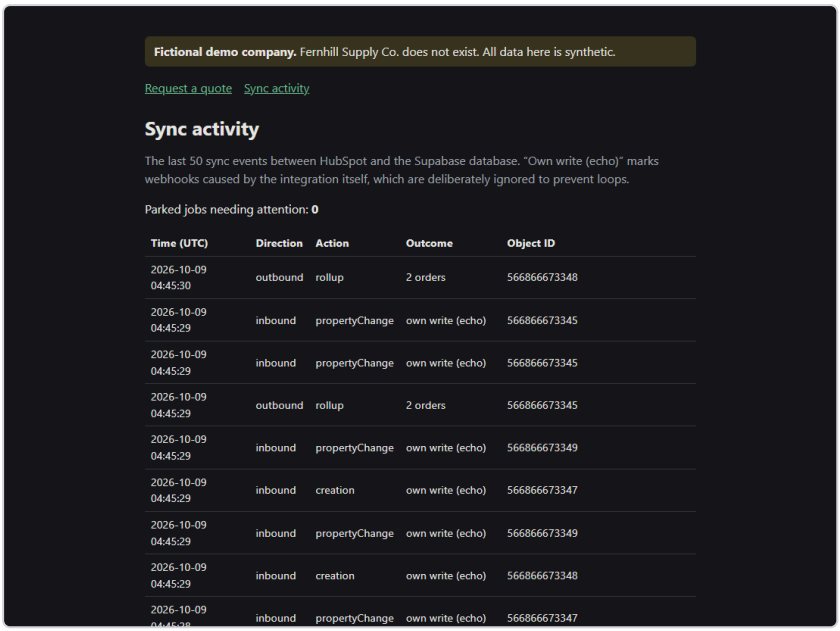
## Screenshots




*The Orders card on a HubSpot contact record.*




*The public quote form.*

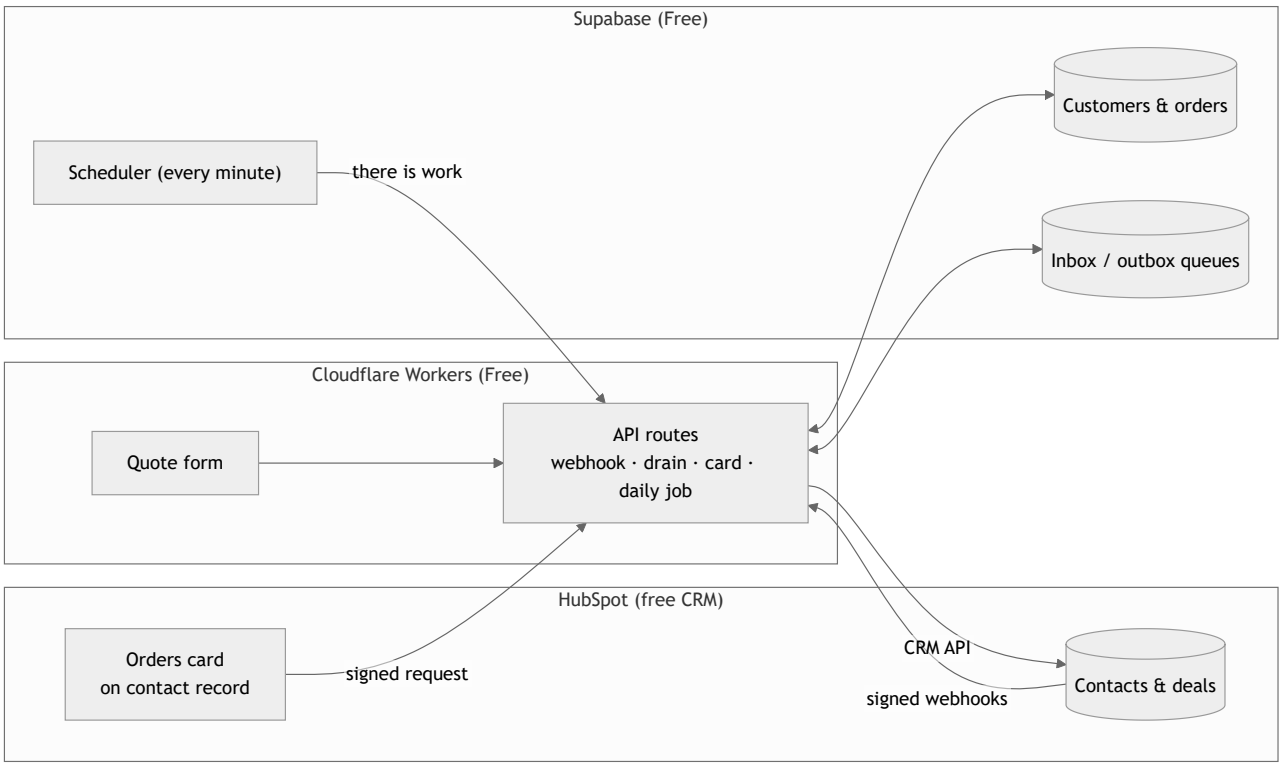



*The sync activity page, captured on 9 October 2026, showing a demo reset.*

# 2. Architecture

This section explains how the system works in plain language. A short technical appendix follows for developers; the full design is in the source code at `docs/ARCHITECTURE.md`, and the requirements at `docs/PRD.md`.

## The big picture




## The three components

| Component | Role | What lives there |
|---|---|---|
| **HubSpot app** "Fernhill Order Sync (demo)" | Connects to the CRM. Sends a notification (webhook) when a contact changes, and shows the **Orders** card on contact records. | App settings, webhook subscriptions, the card, three contact properties: *Demo: total orders*, *Demo: lifetime value*, *Demo: last order date* |
| **Cloudflare Workers** (website and server) | Does all the work: receives HubSpot notifications, runs the sync, serves the quote form, the activity page and the card's data, and runs a daily check. | The Worker (TypeScript, no framework), its daily Cron Trigger and its configuration (secrets) |
| **Supabase** (database) | Stores customers and orders, and two work queues. Signals the Worker every minute (and immediately on new work) when there is something to do. | Postgres database, scheduler (`pg_cron`), one secret in Supabase Vault |

The code for all three lives in one GitHub repository; every push runs automated tests and a secret scan.

## How data flows

### HubSpot → database (contact changes)

1. Someone edits a contact in HubSpot.

2. HubSpot sends a signed notification to the Worker. The Worker checks the signature, saves the notification in a queue and replies at once.
3. The Worker then asks HubSpot for the contact's **current** details and saves them in the database. It never trusts the notification's own copy, so late or out-of-order notifications cannot overwrite newer data.

Deleting a contact in HubSpot clears that customer's personal details in the database; their orders are kept.

### Database → HubSpot (orders)

1. An order is added, changed or removed in the database.
2. In the same database transaction, a job is queued to recalculate that customer's totals, and the Worker is signalled.
3. The Worker recalculates total orders, lifetime value (cancelled orders excluded) and last order date, and updates the three properties on the HubSpot contact.

A customer created directly in the database is linked to the HubSpot contact with the same email, or a new contact is created for them.

### Website → HubSpot (quote form)

The form finds or creates the contact by email and creates a deal linked to it. It records each finished step, so a retry after a failure continues where it stopped.

## Why it is safe and reliable

**No sync loops.** A loop happens when a change made by the integration comes back as a notification and is written again. Three rules prevent this:

- **Each field has one owner.** HubSpot owns name, email, phone and lifecycle stage. The database owns orders and the three order totals. Only the owner's changes are synced.
- **The integration recognises its own writes.** When HubSpot reports a change made by this app, it is logged as "own write (echo)" and ignored.
- **The order totals are not watched.** HubSpot sends no notifications for the three order-total properties, so updating them can never trigger another sync.

**No duplicates (idempotency).** Every HubSpot notification has a unique key, and a notification received twice is stored once and logged as "duplicate ignored". Order jobs for the same customer merge while they wait. Quote-form submissions are recognised for 24 hours, so a reload or double click returns the first result.

**Self-healing.**

- A failed job is retried automatically after 1, 5, 30 and then 120 minutes.
- After 8 failed attempts, or an error that cannot succeed (such as a rejected request), the job is **parked**. Parked jobs are counted on the activity page for a person to review.
- A **daily job** re-checks every contact HubSpot changed in the last 48 hours and confirms every linked contact still exists, which catches anything a lost notification would have missed.

**Security.** Every request from HubSpot is signature-checked (requests older than 5 minutes are refused). Internal endpoints need a secret. The database is closed to public access. Secrets live only in the Worker's encrypted secrets, Supabase Vault and a local file that is never committed.

## Technical appendix

| Topic | Detail |
|---|---|
| Stack | TypeScript on Cloudflare Workers (no framework); Postgres 17 on Supabase with `pg_cron`, `pg_net`, Vault; HubSpot developer platform 2026.09 project with a static-token private app, an app card (React) and webhooks v3 |
| Endpoints | `POST /api/hubspot/webhook` (HubSpot v3 signature), `GET /api/card/orders` (signed `hubspot.fetch`, portal ID checked), `POST /api/drain` (bearer `DRAIN_SECRET`), `GET /api/cron/daily` (bearer `CRON_SECRET`, 06:00 UTC), `POST /api/form` |
| Webhook subscriptions | Contact creation, deletion, privacy deletion, merge, restore; property changes for `firstname`, `lastname`, `email`, `phone`, `lifecyclestage`; HubSpot concurrency capped at 2 |
| Queues | `sync_inbox` (one row per webhook event, unique `dedupe_key`) and `sync_outbox` (`rollup` and `link_contact`, one pending job per customer and kind). Rows are claimed with `FOR UPDATE SKIP LOCKED`; an abandoned claim is reclaimed after 5 minutes. |
| Echo rule | An event with `changeSource = INTEGRATION` and `sourceId = HUBSPOT_APP_ID` is skipped. This is safe because every write the app makes to a HubSpot-owned field is also written to the database in the same handler. |
| Ordering | Contact writes are guarded by HubSpot's `lastmodifieddate`, so an older state never overwrites a newer one. An identical newer state only advances the timestamp (`unchanged`). |
| Rate limits | HubSpot 429 responses are honoured using `Retry-After`. 5xx and network errors are retried 3 times in place, then handed back to the queue. |
| Data rules | Only `@example.com` contacts are stored (demo safeguard). Customers are soft-deleted, except by the demo reset. `sync_log` holds IDs and outcomes only. |
| Data model | `customers`, `orders`, `sync_inbox`, `sync_outbox`, `form_submissions`, `rate_limits`, `sync_log`. Row-level security is on with no policies, and public roles have no grants. |

# 3. Setup guide: deploying from zero

This guide deploys the whole system on **your own** GitHub, Supabase, Cloudflare and HubSpot accounts. Allow about two hours the first time. Each step ends with a **Verify** check, so do not move on until it passes.

The commands are run from a terminal in the repository folder. Commands starting `bash` need a Bash shell (Git Bash on Windows).

## Before you start

**Accounts (all have free plans):**

| Account | Plan | Notes |
|---|---|---|
| GitHub | Free | Holds the code and runs tests on every push |
| Supabase | Free | Database. Choose a region close to your customers (see Limitations) |
| Cloudflare | Workers Free | Hosting (one Worker), the daily scheduled job, and the web address (a domain on your Cloudflare account, or a free `workers.dev` address) |
| HubSpot | Free CRM or higher | You need permission to create apps (Super admin, or the *App Marketplace / Developer* permissions) |

**Software on the computer doing the setup:**

| Tool | Install |
|---|---|
| Node.js 22 | https://nodejs.org |
| Git | https://git-scm.com |
| Docker Desktop (must be running) | https://www.docker.com/products/docker-desktop/. Used to run database migrations and tests. |
| HubSpot CLI | `npm i -g @hubspot/cli` |
| gitleaks (recommended) | `winget install Gitleaks.Gitleaks` or `brew install gitleaks`. Blocks commits that contain secrets. |

> **Windows tip.** If the folder path contains `&`, the `hs` and `npx` shortcuts can fail. Run the CLIs through Node instead, for example `node "%APPDATA%\npm\node_modules\@hubspot\cli\bin\hs.js" project upload`. Wrangler (the Cloudflare CLI) is installed by `npm ci` in `web/`; the npm scripts there already call it through Node.

## Step 1: Get the code

1. In your GitHub account, create a new **private** repository (for example `fernhill-order-sync`) with no README or licence, so it starts empty.
2. Copy the `source/` folder from this delivery to a working folder named `fernhill-order-sync`. Push it to that repository and turn on the secret-scan hook:

```
cd fernhill-order-sync
git init -b main
git add . && git commit -m "Fernhill Order Sync v1.0"
git remote add origin https://github.com/<your-account>/fernhill-order-sync.git
git push -u origin main
git config core.hooksPath scripts/hooks
```

3. Copy `.env.example` to `.env.local`. This file holds your secrets locally and is never committed.

**Verify:** `git status` shows a clean tree and `.env.local` is not listed (it is ignored). On GitHub, the *Actions* tab shows the `ci` and `secret-scan` workflows running.

## Step 2: Supabase (database)

1. In Supabase, create a new project. Save the database password in your password manager.
2. `SUPABASE_DB_URL`: *Connect → Session pooler*. Copy the connection string and put your database password in it.
3. `SUPABASE_SECRET_KEY`: *Project Settings → API Keys → Secret keys*. Create or copy a key starting `sb_secret_`.
4. Put both in `.env.local`, then create the database schema:

```
bash scripts/migrate.sh
```

**Verify:** `bash scripts/migrate.sh --status` lists every migration as `applied`. In *Table Editor* you see `customers`, `orders`, `sync_inbox`, `sync_outbox`, `sync_log`, `form_submissions` and `rate_limits`.

## Step 3: Cloudflare Worker and web address

1. From the `web/` folder: `npm ci`, then sign Wrangler in to your Cloudflare account: `node node_modules/wrangler/bin/wrangler.js login`.
2. Choose the address. In `web/wrangler.jsonc`, set `routes` to a hostname on a domain in your Cloudflare account (for example `orders.example.com`; Cloudflare creates the DNS record on deploy). Or remove `routes` and use the free `https://fernhill-order-sync.<your-subdomain>.workers.dev` address.
3. If your address is not `fernhill.mediaandsoftwaremanager.com`, replace it in these three files and commit:
   - `hubspot/src/app/app-hsmeta.json` (`permittedUrls.fetch`, documentation and support URLs)
   - `hubspot/src/app/webhooks/webhooks-hsmeta.json` (`targetUrl`)
   - `hubspot/src/app/cards/OrdersCard.tsx` (`API_BASE`)

**Verify:** `npm run build` in `web/` ends with a `Total Upload: … gzip: …` line (a dry run that deploys nothing).

## Step 4: HubSpot app

1. Sign the CLI in to your HubSpot account: `hs account auth` (it asks for a personal access key, created in HubSpot under *Development → Keys*).
2. Upload the app from the `hubspot/` folder: `hs project upload`. The first upload to a new account may need `--force`.
3. Install it: *Development → Projects → fernhill-order-sync → app → Distribution → Install.*
4. Copy four values into `.env.local`:
   - `HUBSPOT_TOKEN` is the access token on the app's *Distribution* tab.

- `HUBSPOT_CLIENT_SECRET` is the client secret on the *Auth* tab.
- `HUBSPOT_APP_ID` is the app ID on the *Auth* tab.
- `HUBSPOT_PORTAL_ID` is your HubSpot account ID, shown in the account menu (top right).

5. Create the three order-total contact properties:

```
node scripts/setup-hubspot-properties.mjs
```

**Verify:** the script prints `created` (or `exists`) for the group and all three properties. In HubSpot, *Settings → Properties → Contact properties* shows the group **Demo order data**.

## Step 5: Configure and deploy the web app

1. Copy the configuration into the Worker:

```
node scripts/push-worker-secrets.mjs
```

   This generates `DRAIN_SECRET`, `CRON_SECRET` and `RATE_LIMIT_SALT` into `.env.local` if they are missing. It then uploads every variable as an encrypted Worker secret (`wrangler secret bulk`). It prints names and `ok` only, never values.
2. Deploy from `web/`: `npm run deploy`. This also attaches your address and the daily Cron Trigger (06:00 UTC).

**Verify:** your web address shows the quote form, and `/activity` shows "No sync activity yet."

## Step 6: Connect the database to the web app

Store the work-signal address and secret in Supabase Vault:

```
bash scripts/set-vault-secrets.sh https://<your-domain>/api/drain
```

**Verify:** the script prints `vault secrets set: 2`.

## Step 7: Load demo data and add the card

1. Seed the demo data (synthetic contacts in HubSpot and matching customers and orders in Supabase):

```
node scripts/demo-reset.mjs --yes
```

2. In HubSpot: *Settings → Objects → Contacts → Record customization*. Add the **Orders (Fernhill demo)** card to the contact record.

**Verify:** `/activity` lists `rollup` lines. Opening the contact *Avery Lindqvist* shows the Orders card and the three order totals.

## Step 8: End-to-end check

Run the loop-safety smoke test (about two minutes, including one edit you make in HubSpot when asked):

```
node scripts/smoke-test.mjs --yes
node scripts/demo-reset.mjs --yes
```

**Verify:** the script ends with `PASS: all 16 checks`. If it does not, see the Operations runbook.

## Every environment variable

None of these values may be committed. `.env.local` is gitignored and the secret scan blocks accidental commits.

| Name | Purpose | Where to get it | Stored in | Secret? |
|---|---|---|---|---|
| `HUBSPOT_TOKEN` | Authorises every HubSpot API call the app makes | HubSpot app → *Distribution* tab → access token | `.env.local`, Worker | **Yes** |
| `HUBSPOT_CLIENT_SECRET` | Verifies that webhooks and card requests really come from HubSpot | HubSpot app → *Auth* tab → client secret | `.env.local`, Worker | **Yes** |
| `HUBSPOT_APP_ID` | Recognises the app's own writes, so they are not synced back (loop prevention) | HubSpot app → *Auth* tab → app ID | `.env.local`, Worker | No |
| `HUBSPOT_PORTAL_ID` | Card requests from any other HubSpot account are refused | HubSpot account menu → account ID | `.env.local`, Worker | No |
| `SUPABASE_SECRET_KEY` | Server-side database access (bypasses row-level security; never sent to a browser) | Supabase → *Project Settings* → *API Keys* → *Secret keys* | `.env.local`, Worker | **Yes** |
| `SUPABASE_DB_URL` | Direct database connection for migrations and Vault setup (scripts only) | Supabase → *Connect* → *Session pooler* | `.env.local` only | **Yes** (contains the database password) |
| `SUPABASE_URL` | Address of the Supabase API | Derived automatically from `SUPABASE_DB_URL`; set it only to override | Worker | No |
| `DRAIN_SECRET` | Lets Supabase tell the web app that work is waiting | Generated by `push-worker-secrets.mjs` | `.env.local`, Worker, Supabase Vault (`drain_secret`) | **Yes** |
| `CRON_SECRET` | Authorises the daily job call | Generated by `push-worker-secrets.mjs`; the Cron Trigger sends it automatically | `.env.local`, Worker | **Yes** |
| `RATE_LIMIT_SALT` | Hashes visitor IP addresses for form rate limits, so raw IPs are never stored | Generated by `push-worker-secrets.mjs` | `.env.local`, Worker | **Yes** |

Supabase Vault also holds `drain_url`, the address of `/api/drain`, set in Step 6.

**Changing a Worker secret takes effect at once:** uploading secrets deploys a new version of the Worker.

# 4. Operations runbook

## Daily monitoring (2 minutes)

Open **https://fernhill.mediaandsoftwaremanager.com/activity** (or `/activity` on your own address).

| Check | Healthy | Action if not |
|---|---|---|
| **Parked jobs needing attention** | `0` | See Parked jobs |
| Latest `system / daily` line | One per day, ending `0 errors` | See What the daily job does |
| Recent `inbound` and `outbound` lines | Outcomes such as `updated`, `unchanged`, `N orders`, `own write (echo)`, `duplicate ignored` | An `error` outcome means a job failed. It retries automatically; act only if it ends up parked. |
| Page loads | The table shows | "The activity log is unavailable" usually means the database is paused or unreachable. See Supabase paused. |

**What the outcomes mean:** `updated` means the database took HubSpot's newer data. `unchanged` means nothing differed. `stale` means an older notification arrived late and was correctly ignored. `own write (echo)` is the integration's own change coming back, ignored by design. `N duplicates ignored` means a notification was received again and de-duplicated. `skipped` means there was nothing to do, for example a deleted customer.

**Logs:** Cloudflare → *Workers & Pages* → `fernhill-order-sync` → *Logs* (or `wrangler tail` from `web/`). The app writes one JSON line per request or job (for example `webhook_received`, `drain`, `job_failed`, `webhook_rejected`, `daily`) and never logs names, emails or secrets.

## What the daily job does

The Worker's Cron Trigger runs the daily job (`/api/cron/daily`) once a day at **06:00 UTC**. The job:

1. Deletes activity lines and finished queue rows older than 30 days, and form rate-limit records older than 2 days.
2. Retries any queued work that is due.
3. Re-checks every contact HubSpot modified in the last **48 hours** and applies its current state. This repairs anything a lost notification missed.
4. Confirms every linked contact still exists in HubSpot. Deleted ones are soft-deleted in the database, and merged ones are re-pointed to the surviving contact.
5. Writes one `daily` line to `/activity`, such as `12 recent, 8 checked, 0 errors`.

If the line shows errors, check the Worker logs for `daily_contact_failed` (it names the contact ID and the error). If it shows `timed out`, the run hit its time limit or the Workers limit of 50 outgoing requests per run; the next day's run continues the work, and the per-minute drain keeps processing queued work in between.

Separately, Supabase's scheduler signals the web app **every minute** so that queued work and retries are processed. This also keeps the free Supabase project active.

# Common issues

## Supabase project paused

- **Symptoms:** `/activity` says the log is unavailable. Worker logs show `webhook_insert_failed`, and the quote form fails.
- **Cause:** free Supabase projects pause after 7 days without activity. The per-minute scheduler is designed to prevent this, but a paused project stops the scheduler too.
- **Fix:** Supabase dashboard → project → **Restore project**, then wait a few minutes.
- **Afterwards:** HubSpot retries notifications that failed with a server error for up to 24 hours, and the daily job repairs changes from the last 48 hours. If the project was paused longer than 48 hours, edit or re-save the affected contacts. For a demo, `node scripts/demo-reset.mjs --yes` restores everything.

## HubSpot token expired, revoked or rotated

- **Symptoms:** jobs fail with `HTTP 401`, are parked immediately (a rejected request cannot succeed on retry), and the parked count rises. The card still loads, because it reads only from the database.
- **Fix:** get the current token from the HubSpot app's *Distribution* tab. Update `HUBSPOT_TOKEN` in `.env.local` and run `node scripts/push-worker-secrets.mjs` (it takes effect at once). Then re-queue the parked jobs.

## Webhook signature failures

- **Symptoms:** HubSpot edits stop reaching the database. Worker logs show `webhook_rejected` with a reason.
- **Reasons and fixes:**
  - `signature mismatch`: `HUBSPOT_CLIENT_SECRET` in the Worker is wrong or old. Update it from the app's *Auth* tab and run `node scripts/push-worker-secrets.mjs`. If the app was moved to a new web address, the webhook `targetUrl` and the address HubSpot calls must match.
  - `timestamp outside 5 min window`: an old request was replayed, or the sender's clock is wrong. Genuine HubSpot traffic is never affected.
  - `missing signature headers`: the request did not come from HubSpot. Nothing to fix.
- **Afterwards:** do not rely on HubSpot redelivering notifications that were rejected. The daily job re-checks every contact changed in the last 48 hours, so it repairs anything missed in that window. For a longer outage, re-save the affected contacts.

## Parked jobs after 8 attempts

- **Where to look:** Supabase → *Table Editor* → `sync_outbox` or `sync_inbox`, filtered to `status = dead`. The `last_error` column says why.
- **Typical causes:** an expired token (see above), a contact whose email is already used by another linked customer, or a customer outside the demo email domain.
- **Fix the cause first**, then re-queue the jobs in *SQL Editor*:

```
update public.sync_outbox set status = 'pending', attempts = 0, next_attempt_at = now(),
last_error = null
 where status = 'dead';
update public.sync_inbox  set status = 'pending', attempts = 0, next_attempt_at = now(),
last_error = null
 where status = 'dead';
```

The per-minute signal picks them up. Check that the parked count on `/activity` returns to 0.

### Rate limits

- **HubSpot API:** the app waits as HubSpot asks when it is rate-limited (HTTP 429), then hands the job back to the queue with back-off. You will see `rate limited` errors that clear by themselves. HubSpot's free limits are listed in Limitations.
- **Quote form:** each visitor can submit 5 times an hour, and the whole site 200 times a day. Raise these limits in `web/app/api/form/route.ts` (`PER_IP_PER_HOUR`, `SITE_PER_DAY`) if real traffic needs it.

## Demo reset

```
node scripts/demo-reset.mjs --yes
```

This archives every `@example.com` contact and every `Fernhill demo:` deal in HubSpot, deletes all demo data in Supabase, then reseeds 6 synthetic contacts with 13 orders. It takes about 5 seconds and reseeded contacts get new HubSpot IDs. **It is for the demo only.** Never run it against an account holding real customers.

## Rotating secrets

Rotate a secret straight away if it may have leaked, and otherwise on a schedule you choose (for example yearly). Update the value in `.env.local` and run `node scripts/push-worker-secrets.mjs`: Worker secrets take effect at once (the upload deploys a new version).

| Secret | How to rotate | Side effects |
|---|---|---|
| `HUBSPOT_TOKEN` | HubSpot app → *Distribution* tab → rotate the access token. Update `.env.local`, run `push-worker-secrets.mjs`. | Jobs fail until the upload, then retry. Re-queue any parked jobs. |
| `HUBSPOT_CLIENT_SECRET` | HubSpot app → *Auth* tab → rotate the client secret. Update, push. | Webhooks are rejected until the upload. The next daily job repairs contacts changed meanwhile. |
| `SUPABASE_SECRET_KEY` | Supabase → *API Keys* → create a new secret key. Update, push, then delete the old key. | None if the old key is deleted only after the upload. |
| Database password (in `SUPABASE_DB_URL`) | Supabase → *Database settings* → reset the password. Update `.env.local`. | Used by scripts only; the running app is unaffected. |
| `DRAIN_SECRET` | Delete its line from `.env.local` and run `push-worker-secrets.mjs` (a new one is generated), then run `bash scripts/set-vault-secrets.sh https://<your-domain>/api/drain`. | Work signals are refused for a minute or two between the steps. Queued work waits and is not lost. |
| `CRON_SECRET` | Delete its line, run `push-worker-secrets.mjs`. | None (the Cron Trigger sends the new value automatically). |
| `RATE_LIMIT_SALT` | Delete its line, run `push-worker-secrets.mjs`. | Current form rate-limit counters reset. |
| HubSpot personal access key (CLI) | HubSpot → *Development* → *Keys*. Revoke, create a new one, run `hs account auth`. | CLI only. |

After a rotation, check `/activity` and run the smoke test if in doubt.

# 5. Access and ownership

This is the inventory of every account, app, key and secret in the system. It gives **names and locations only, never values**. Today everything is owned by Media & Software Manger. The client takes ownership by deploying their own copy; the checklist at the end tracks it.

## Accounts and services

| # | Item | Platform / plan | Identifier | Current owner | Holds |
|---|---|---|---|---|---|
| A1 | Source code | Git repository (developer's GitHub account) | Delivered as the `source/` folder | Media & Software Manger | All code, documentation, CI workflows ( `ci` , `secret-scan` ). **No secrets**: the workflows need none. |
| A2 | Hosting | Cloudflare Workers (Free) | Worker `fernhill-order-sync` , address `fernhill.mediaandsoftwaremanager.com` (custom domain on the developer's zone), code in `web/` | Media & Software Manger's Cloudflare account | The web app, 9 Worker secrets (B1–B9), the daily Cron Trigger. Deployed with `npm run deploy` from `web/` |
| A3 | Database project | Supabase (Free) | Project `fernhill-order-sync` , organisation `MSmanager` , region Northeast Asia (Seoul) | Media & Software Manger | All tables and data, Vault secrets (C1–C2), the per-minute scheduler job `drain-every-minute` |
| A4 | CRM account | HubSpot (free CRM) | Account "media and software company" | Media & Software Manger | Demo contacts and deals, the app (A5), the property group *Demo order data* |
| A5 | HubSpot app | HubSpot developer project (platform 2026.09) | Project `fernhill-order-sync` , app "Fernhill Order Sync (demo)" (private, static token), card "Orders (Fernhill demo)", webhook subscriptions | Lives inside A4 | Access token (B1), client secret (B2), app ID (B3) |
| A6 | Local configuration | Developer computer | `.env.local` in the repository folder (gitignored) | Media & Software Manger | Copies of every secret below, used by the setup and operations scripts |

## Secrets and keys

| # | Name | Where it lives | Owner | Rotate via |
|---|---|---|---|---|
| B1 | `HUBSPOT_TOKEN` | Worker, `.env.local` | HubSpot app A5 | Runbook: rotating secrets |
| B2 | `HUBSPOT_CLIENT_SECRET` | Worker, `.env.local` | HubSpot app A5 | Runbook |

| # | Name | Where it lives | Owner | Rotate via |
|---|---|---|---|---|
| B3 | `HUBSPOT_APP_ID` (not secret) | Worker, `.env.local` | HubSpot app A5 | Changes only if the app is re-created |
| B4 | `HUBSPOT_PORTAL_ID` (not secret) | Worker, `.env.local` | HubSpot account A4 | Changes only with the HubSpot account |
| B5 | `SUPABASE_SECRET_KEY` | Worker, `.env.local` | Supabase project A3 | Runbook |
| B6 | `SUPABASE_URL` (not secret) | Worker | Supabase project A3 | Changes only with the project |
| B7 | `DRAIN_SECRET` | Worker, `.env.local`, Supabase Vault (C2) | Generated | Runbook |
| B8 | `CRON_SECRET` | Worker, `.env.local` | Generated | Runbook |
| B9 | `RATE_LIMIT_SALT` | Worker, `.env.local` | Generated | Runbook |
| B10 | `SUPABASE_DB_URL` (contains the database password) | `.env.local` only | Supabase project A3 | Supabase → *Database settings* → reset password |
| C1 | Vault secret `drain_url` (not secret) | Supabase Vault | Supabase project A3 | `scripts/set-vault-secrets.sh` |
| C2 | Vault secret `drain_secret` | Supabase Vault | Supabase project A3 | `scripts/set-vault-secrets.sh` |
| D1 | HubSpot personal access key (CLI login) | The developer's HubSpot CLI configuration | The developer's HubSpot user | HubSpot → *Development* → *Keys* |
| D2 | Cloudflare (Wrangler) and GitHub CLI logins | The developer's computer | The developer | Each platform's account settings |

All nine Worker secrets were set on 22 October 2026 (names checked, not values). Worker secrets are encrypted and cannot be read back, only replaced.

## Transferring ownership

**Recommended: a fresh deployment on the client's own accounts.** The live system holds only synthetic demo data, so there is nothing to migrate. The client follows the Setup guide (about two hours) on their own GitHub, Supabase, Cloudflare and HubSpot accounts. Every secret is then created fresh by the client and never known to the developer. Once the client's deployment passes the smoke test, the developer decommissions the demo on an agreed date (see Decommissioning the demo).

### Source code (A1)

The source code is delivered as a plain folder, so nothing needs to be transferred out of the developer's account.

1. The client creates a **private** repository in their own GitHub account or organisation.
2. The client pushes the `source/` folder to that repository (Setup guide, Step 1). It contains the code only: no git history and no credentials.
3. GitHub Actions runs the `ci` and `secret-scan` workflows on the first push. They need no configuration or secrets.
4. From then on, the client's repository is the source of truth.

### Supabase (A3) and Cloudflare (A2)

The client creates a new Supabase project and a new Worker (Setup guide Steps 2, 3, 5 and 6) on a domain in their own Cloudflare account, and updates the three references listed in Setup guide Step 3. The demo address `fernhill.mediaandsoftwaremanager.com` belongs to the developer's domain and is not transferred.

> Supabase can also move an existing project between organisations (*Project Settings → General → Transfer project*). That requires the developer and the client to be members of each other's organisation during the move. A Worker is simply redeployed from the source code. It brings nothing a fresh deployment lacks, so it is not the recommended route.

### HubSpot app (A4, A5)

The app is defined entirely in the source code (`hubspot/`). A HubSpot project belongs to the account it is uploaded to, so the client uploads it into **their own HubSpot account** (Setup guide Step 4). That installs the app, creates the three properties and adds the card. Contacts and deals stay in whichever account they were created in.

### Decommissioning the demo

After the client confirms their deployment works, the developer:

1. Deletes the demo Worker `fernhill-order-sync` and its custom domain (stops the site and the daily job).
2. Deletes the demo Supabase project (removes all synthetic data and the Vault secrets).
3. Uninstalls the demo app in the developer's HubSpot account and archives the demo contacts and deals.
4. Deletes the local `.env.local` and confirms this in writing.

## Transfer checklist

- [ ] Source code pushed to the client's own private GitHub repository; both workflows pass
- [ ] Supabase project created in the client's account; migrations applied; Vault secrets set
- [ ] Worker deployed from `web/` to the client's Cloudflare account, on the client's own address; secrets uploaded
- [ ] HubSpot app uploaded and installed in the client's account; properties created; Orders card added
- [ ] All secrets (B1–B10, C1–C2) created by the client and held only in the client's accounts
- [ ] Smoke test passes on the client's setup (`node scripts/smoke-test.mjs --yes`)
- [ ] `/activity` shows 0 parked jobs and a `daily` line from the client-owned deployment
- [ ] Demo decommissioned by the developer on the agreed date, and `.env.local` deletion confirmed in writing

# 6. Testing and QA

Three layers of checks protect the system: **automated tests** on every code change, **secret scanning** on every commit, and a **live smoke test** against the real deployment.

| Layer | Count | Runs |
|---|---|---|
| Unit tests (web app) | **54** tests in 7 files | Every push (GitHub Actions) and locally: `cd web && npm test` |
| Database checks | **77** checks | Every push, against Supabase's own Postgres image; locally: `bash supabase/tests/run.sh` (needs Docker) |
| Type checks and production build | Web app and HubSpot card | Every push |
| Secret scan | Full git history | Every push (CI), and on every commit (local hook) |
| Live smoke test | 16 checks | On demand: `node scripts/smoke-test.mjs --yes` |

> The first release candidate had 51 unit tests and 71 database checks. On 9 October 2026, 3 unit tests and 6 database checks were added with the fixes from the first smoke test (described below).

## What the unit tests cover (54)

| Area | What is proven |
|---|---|
| **Request signatures** (7) | Valid HubSpot v3 signatures are accepted for GET and POST. Tampered queries, stale or future timestamps, missing headers and wrong-length signatures are rejected. Internal endpoints accept only the exact secret. |
| **HubSpot client** (7) | Rate limits are honoured ( `Retry-After` ). 5xx and network errors are retried up to 3 times. Other 4xx errors fail permanently. Find-or-create never overwrites an existing contact, and a parallel create is resolved to the winner. |
| **HubSpot → database** (12) | The echo rule drops the app's own writes; other integrations' writes are processed. Current state is fetched rather than trusting the event. Non-demo contacts are never stored. Deletes, merges and 404s are handled, and the de-duplication key is correct. |
| **Database → HubSpot** (7) | Roll-ups patch the three properties correctly. Unlinked customers wait (with a 24-hour safety limit). Linking never overwrites a HubSpot name. Out-of-domain customers are refused. |
| **Job runner** (2) | Successes finish; errors fail as permanent or retryable; rows that cannot start before the deadline are handed back. |
| **Quote form** (9) | Validation (demo emails only), duplicate detection, step-by-step resume after failure, reuse of a deal created by an earlier attempt, and no HubSpot calls for a completed duplicate. |
| **Daily job** (3) | Paging through recent contacts, resolving missing ones, isolating a failing contact, pruning even when out of time. |
| **Routes** (7) | Every endpoint rejects unauthenticated requests before touching the database. The webhook logs duplicates as ignored. |

## What the database checks cover (77)

| Area | What is proven |
|---|---|
| **Access control** | The public ( `anon` ) role cannot read data or call functions; only the server role can. |

| Area | What is proven |
|---|---|
| **Queueing** | App-created customers queue a link job; HubSpot-mirrored ones queue nothing (loop safety). A burst of order changes merges into one job. Customers with orders cannot be hard-deleted. |
| **Job lifecycle** | Claiming, no double claims, reclaiming stale work, back-off, parking after the final attempt, re-queue and release. |
| **Contact sync rules** | Insert, update, stale (older) data ignored, unchanged data only advances the timestamp, linking by email, email conflicts, soft delete clears personal data, restore, merge, and a late delete never overriding a later restore. |
| **Roll-ups and card** | Cancelled orders are excluded; the card query pages correctly. |
| **Form and rate limits** | Duplicate submissions, in-progress protection, resume timing, 24-hour window, rate-limit blocking. |
| **Scheduler and Vault** | The work signal uses the Vault address and secret, is a silent no-op without them, and is scheduled every minute. |
| **Retention, reset and seed** | Pruning old rows; the demo reset leaves nothing behind; seeding queues one roll-up per customer and no link work. |

## Continuous integration and secret scanning

- `ci` **workflow** (GitHub Actions, every push and pull request): unit tests, type check and production build of the web app; type check of the HubSpot card; migrations and database checks against `supabase/postgres:17.11.0.004`.
- `secret-scan` **workflow:** gitleaks 8.30.1 scans the **entire git history** and prints any findings redacted.
- **Local pre-commit hook:** gitleaks scans staged changes before every commit (`git config core.hooksPath scripts/hooks`).
- **Latest results:** commit `7706ac3` (9 October 2026, 04:45 UTC): `ci` **passed** and `secret-scan` **passed**. The three previous commits passed both workflows too.

## Live smoke test results

Full evidence, with timestamps: `docs/SMOKE-TEST.md` in the source code.

### Run 1: 9 October 2026, 04:20–04:25 UTC. PASS (5 of 5 steps)

| Check | Result |
|---|---|
| A human edit in HubSpot (phone number) reached the database once, with no write back to HubSpot | PASS (database updated 3.4 s after the edit) |
| One new order caused exactly one HubSpot update of the three totals, one history entry each, and no echo | PASS |
| Replaying an already-processed webhook was de-duplicated; a stale signature was refused | PASS |
| Demo reset restored the seed state | PASS |

This run found four improvements (not defects): duplicates were invisible on `/activity`, `/activity` used different IDs per direction, contact rows were rewritten without change, and there was no repeatable smoke script. All four were fixed, tested and deployed the same day.

### Run 2, after the fixes: 9 October 2026, 04:42–04:45 UTC. PASS (16 of 16 checks)

Run with the new `scripts/smoke-test.mjs` against production (commit `2a5f802`):

| Step | Result |
|---|---|
| 1. HubSpot UI edit (last name) → database | 5/5: one event, database updated, shown on `/activity`, zero outbound jobs, zero HubSpot writes |
| 2. New order → HubSpot | 6/6: one roll-up job, correct values (3 → 4 orders, 1080.29 → 1203.74), one history entry each, no echo, `/activity` shows the HubSpot contact ID, cleanup restored the totals |
| 3. Idempotency | 5/5: replay accepted but not stored again, "1 duplicate ignored" on `/activity`, nothing re-processed, stale signature refused (401) |
| Restore | Demo reset completed, 0 parked jobs |

## How to re-test after any change

1. `cd web && npm test && npm run typecheck`
2. `bash supabase/tests/run.sh` (if SQL changed; apply with `bash scripts/migrate.sh`)
3. Push, and confirm both GitHub Actions workflows pass.
4. `node scripts/smoke-test.mjs --yes`, then `node scripts/demo-reset.mjs --yes`.

# 7. Limitations and next steps

The system was built as a **demonstration on free tiers, with no paid plan and no credit card**. It is production-grade in design (signed requests, queues, retries, idempotency, tests), but the free plans and the demo safeguards must change before it handles real customers.

## Free-tier constraints

| Platform | Constraint | Effect today | Mitigation in place |
|---|---|---|---|
| **Supabase Free** | Projects **pause after 7 days of inactivity**. 500 MB database. 2 active projects per organisation. **No backups** on the live project. | A paused project stops all syncing until it is restored by hand. | The per-minute scheduler and the daily job keep the project active. The runbook explains how to restore. |
| **Cloudflare Workers Free** | 100,000 requests a day; **10 ms of CPU** and **50 outgoing requests** per request or scheduled run; background work ends **30 s** after the response; logs kept only briefly. | Ample for the demo. A burst of many queued changes is processed over several runs instead of one. | No framework (pages are plain HTML). Each drain stops in time and hands remaining work to the next run, which comes within a minute. |
| **Cloudflare Cron Trigger** | The daily job is one scheduled run, with the same 50-request limit. | The safety-net check runs daily (06:00 UTC); with many contacts it continues the next day (`timed out` on `/activity`). | Minute-level retries are driven by Supabase's scheduler. |
| **HubSpot API (free CRM, private app)** | 100 requests per 10 seconds and 250,000 per day per app. The search API has a lower limit. Webhooks time out after 5 s and are retried up to 10 times over 24 hours. | Ample for the demo. A large import could hit the 10-second limit. | Rate-limit responses are honoured, webhook concurrency is capped at 2, and jobs run one at a time. |
| **HubSpot free features** | Custom objects, HubSpot-hosted serverless functions and workflows need paid hubs. | Orders are shown with a card and three contact properties rather than as a HubSpot object. | — |

## Demo safeguards to remove or change for real data

These protect the demo. They must be changed deliberately when real customers are involved:

- **The `@example.com` rule.** The sync stores only contacts with `@example.com` emails (`DEMO_EMAIL_DOMAIN` in `web/lib/hubspot.ts`). The form accepts only those emails, and the daily job clears anything else. For real use, replace it with the client's own rule: which contacts to sync (for example a list or lifecycle stage).
- **The demo reset** archives every `@example.com` contact and wipes the database. Remove `scripts/demo-reset.mjs` (or guard it) in a real deployment.
- **Property names** (`demo_total_orders` and the others) and the "(demo)" app and card names should be renamed.
- **The public activity page** shows IDs and outcomes only, but a business may prefer it behind a login.
- **Fernhill branding** and the "fictional company" banner on the website.

## What changes for real customer data

| Topic | Recommendation |
|---|---|
| **Paid tiers** | Supabase Pro (no pausing, daily backups, more capacity) and Cloudflare Workers Paid ($5 a month: more CPU time and 10,000 outgoing requests per run, longer log retention). HubSpot: the client's existing subscription; Operations Hub or Enterprise only if custom objects or workflows are wanted. |
| **Privacy and DPA** | Sign each vendor's Data Processing Addendum (HubSpot, Supabase, Cloudflare). Record the integration in your records of processing. Update the privacy notice for the website form. Set a retention policy for orders and for the `sync_log`; the current 30-day log pruning is a starting point. |
| **Data region** | The demo database is in **Northeast Asia (Seoul)**, and the Worker runs in Cloudflare's network, near whoever calls it. For EU or UK customer data, create the Supabase project in an EU region; the Worker stores no data itself. Region cannot be changed after a Supabase project is created, so plan a migration. |
| **Backups and recovery** | Turn on Supabase backups (Pro: daily; Point-in-Time Recovery as an add-on), and test a restore once. HubSpot remains the system of record for contacts; the database is the system of record for orders. |
| **Access and security** | Use organisation-owned accounts (not personal ones) with 2-factor sign-in. Rotate secrets on a schedule (see the runbook). Add a GitHub branch protection rule so that `main` requires passing CI. |
| **Monitoring** | Add an alert when the parked-job count is above zero or the daily job reports errors (for example a small uptime or log-alert service), rather than relying on someone checking `/activity`. |

## Upgrade roadmap (prioritised)

| Priority | Item | Why |
|---|---|---|
| **1. Before any real data** | Deploy on client-owned accounts, with all secrets created by the client (Access and ownership) | Ownership and security |
| **1** | Supabase Pro in the right region, with backups on | No pausing, recoverability, data residency |
| **1** | Cloudflare Workers Paid | Headroom for real data volumes (CPU and outgoing requests per run) |
| **1** | Replace the `@example.com` rule and remove the demo reset | The demo safeguards would block or delete real data |
| **1** | DPAs signed and privacy notice updated | Legal basis for processing |
| **2. First month** | Alerting on parked jobs and daily-job errors | Problems found in minutes, not days |
| **2** | Activity page behind a login, or internal only | Least exposure |
| **2** | Rename the demo properties, app and card | Clean user experience in HubSpot |
| **3. Later** | Sync deals or other objects back to the database; richer order details on the card | Business features, depending on need |
| **3** | Run the reconciliation more often (a second Cron Trigger) | Faster self-healing after outages |

# 8. Handoff checklist and sign-off

Tick each item once you have seen it working. Items marked **(live)** can be checked on the public deployment today; the others need the client's own accounts.

## Functionality

- [ ] **(live)** The quote form at https://fernhill.mediaandsoftwaremanager.com creates a contact and a deal in HubSpot, and a second identical submission shows "Already received" without creating duplicates.
- [ ] **(live)** The **Orders** card on a HubSpot contact lists that contact's orders, newest first, and shows an empty state for a contact with no orders.
- [ ] **(live)** Editing a contact's name, email, phone or lifecycle stage in HubSpot updates the database within 2 minutes.
- [ ] **(live)** Adding an order in the database updates the contact's total orders, lifetime value and last order date in HubSpot within 2 minutes.
- [ ] **(live)** Deleting a demo contact in HubSpot clears its personal details in the database and keeps its orders.
- [ ] **(live)** https://fernhill.mediaandsoftwaremanager.com/activity lists sync events with no names or emails, and shows "Parked jobs needing attention: 0".

## Reliability and safety

- [ ] **(live)** No sync loops: one change produces exactly one write on the other side (smoke test steps 1 and 2).
- [ ] **(live)** Duplicate notifications are ignored and shown as "duplicate ignored" (smoke test step 3).
- [ ] **(live)** A `daily` line appears on `/activity` each day.
- [ ] Unsigned or stale requests to the webhook, card, drain and daily endpoints are refused (covered by unit tests).

## Quality evidence

- [ ] Both GitHub Actions workflows (`ci`, `secret-scan`) pass on the delivered commit.
- [ ] 54 unit tests and 77 database checks pass.
- [ ] The latest smoke test passes all 16 checks (Testing and QA).
- [ ] The handoff package contains no secrets, tokens, real emails or personal data.

## Ownership and documentation

- [ ] Documents 1–7 and this checklist received, as Markdown and as the PDF.
- [ ] Deployed on the client's own GitHub, Supabase, Cloudflare and HubSpot accounts (Access and ownership).
- [ ] All secrets created by the client; the demo deployment decommissioned on the agreed date.
- [ ] The client has run `node scripts/smoke-test.mjs --yes` on their own setup.
- [ ] Limitations and the upgrade roadmap read and accepted (Limitations and next steps).

## Sign-off

By signing, the client accepts delivery of Fernhill Order Sync version 1.0 as described in this package.

| | Client | Media & Software Manger |
|---|---|---|
| Name | | |
| Role | | |
| Date | | |
| Signature | | |

Notes or exceptions: